



SMART CONTRACT SECURITY ASSESSMENT

Bankroll vltUSDC Vault

SECURITY REVIEW

19 JULY 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1	About Shieldify	3
2	Disclaimer	3
3	About Bankroll (vltUSDC Vault)	3
3.1	Overview	3
4	Risk Classification	4
4.1	Impact	4
4.2	Likelihood	4
5	Security Review Summary	4
5.1	Protocol Summary	5
5.2	Scope	5
6	Findings Summary	5
7	Findings	6

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About Bankroll (vltUSDC Vault)

3.1 Overview

vltUSDC is a tokenized, auto-compounding liquidity vault built on a single full-range Uniswap V4 VLT/USDC position. Depositors receive `vltUSDC` — a standard ERC-20 share that lives in the holder's own wallet and represents a pro-rata claim on the position's raw liquidity units (`L`). The system holds no price oracle: shares are denominated in liquidity, never in dollars, and redemption returns both underlying tokens in kind.

The vault is designed to be ownerless and non-upgradeable — there is no proxy, no admin role, no pause, and no sweep. Every parameter is a constant or fixed at deployment, and both `deposit` and `redeem` are permissionless. Compounding has no dedicated entry point: once roughly \$100 of trading fees is claimable, the next deposit harvests them, rebalances the vault's inventory within a hard 5% pool-price-movement bound, and reinvests 100% of the harvest as new liquidity, minting no shares so that `L` grows against a fixed supply.

At a high level, the main functionalities are:

- Users deposit VLT and USDC (or USDC-only through the **ZapHelper** periphery) and receive **vltUSDC** shares priced against the liquidity actually added (ΔL), measured from the pool to neutralize donation and first-deposit inflation.
- Holders redeem shares at any time, burning them for a pro-rata, in-kind withdrawal of both underlying tokens with no swap and no forced sale.
- Trading fees earned by the position auto-compound inside the deposit path — no keeper, no compound fee — increasing each share's redemption value over time.
- A periphery **ZapHelper** allows USDC-only entry by routing an off-chain-computed swap into VLT before depositing the pair.

Learn more: bankroll.network

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	 CRITICAL	 HIGH	 MEDIUM
Likelihood: Medium	 HIGH	 MEDIUM	 LOW
Likelihood: Low	 MEDIUM	 LOW	 LOW

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

The security review lasted 5 days, with a total of 80 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying eleven Informational recommendations. They are mostly focused on minor recommendations regarding share accounting and fee accrual behaviour.

The Bankroll team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	Bankroll (vltUSDC Vault)
Repository	bankroll-contracts
Type of Project	Liquidity Vault (Uniswap V4 Auto-Compounding LP)
Security Review Timeline	5 days
Review Commit Hash	4dae465b0ba659b7f224bdf17ba0bcbf6d9eb187
Fixes Review Commit Hash	ce44c31a03b88bd5ac18a7b6d46836e8cf03873c

5.2 Scope

The following smart contracts were in the scope of the security review:

FILE	NSLOC
contracts/VltUsdcVault.sol	432
contracts/ZapHelper.sol	69
Total	501

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Info** issues: 11

ID	TITLE	SEVERITY	STATUS
I-01	New Depositors Capture Fees Earned Before Entry Because Share Minting Excludes Uncompounded Assets	● INFO	ACKNOWLEDGED
I-02	Subsequent Deposits Can Add Liquidity While Minting Zero Shares	● INFO	FIXED

ID	TITLE	SEVERITY	STATUS
I-03	Redeemers Forfeit Their Pro-Rata Share of Uncompounded Fees and Retained Balances	● INFO	ACKNOWLEDGED
I-04	Donation-Inflated Fresh Fees Allow Profitable Sandwiches of the Compound Rebalance	● INFO	ACKNOWLEDGED
I-05	<code>feeApr()</code> Reports All No-Mint Liquidity Growth as Fee APR, Contradicting Its Pitched Yield Source	● INFO	FIXED
I-06	<code>ZapHelper</code> External Swap Functions Lack a Reentrancy Guard	● INFO	FIXED
I-07	<code>zap()</code> Has No Deadline, Allowing Stale Swap Transactions to Execute Later	● INFO	FIXED
I-08	<code>previewRedeem()</code> Can Return Incorrect Quotes for Out-of-Range Share Inputs	● INFO	FIXED
I-09	Position Key Is Recomputed Instead of Cached	● INFO	FIXED
I-10	<code>recipient == vault</code> Permanently Locks Minted Shares	● INFO	FIXED
I-11	Missing Constructor Validation Can Permanently Misconfigure the Vault	● INFO	FIXED

7 Findings

FINDING I-01

New Depositors Capture Fees Earned Before Entry Because Share Minting Excludes Uncompounded Assets

● INFO

ACKNOWLEDGED

Description

Vault shares are priced only against raw Uniswap v4 position liquidity `L`. However, vault-owned value can also exist outside `L` as:

- pending position fees;
- fees harvested and retained during an earlier deposit or redemption;
- token balances left after a partial or zero-liquidity compound; and
- compound dust.

When `compoundClaimable()` reports less than `AUTO_COMPOUND_MIN_USDC`, `deposit()` skips `_compound()`. It then snapshots the vault's existing balances, pulls the depositor's tokens, and opens the deposit callback:

```
[, uint256 claimableUsdc,] = compoundClaimable();
if (claimableUsdc >= AUTO_COMPOUND_MIN_USDC && positionLiquidity() > 0) {
    _compound();
}

uint256 retain0 = _selfBalance(currency0);
uint256 retain1 = _selfBalance(currency1);
```

Inside `_onDeposit()`, a zero-liquidity `modifyLiquidity()` call realizes all pending position fees. The vault correctly excludes those fees from the depositor's liquidity contribution by adding them to `retain0` and `retain1`:

```
[, BalanceDelta fees] = poolManager.modifyLiquidity(
    poolKey,
    ModifyLiquidityParams({
        tickLower: tickLower,
        tickUpper: tickUpper,
        liquidityDelta: 0,
        salt: bytes32(0)
    }),
    ""
);

if (f0 > 0) {
    poolManager.take(currency0, address(this), uint256(uint128(f0)));
    retain0 += uint256(uint128(f0));
}
if (f1 > 0) {
    poolManager.take(currency1, address(this), uint256(uint128(f1)));
    retain1 += uint256(uint128(f1));
}
```

This prevents the historical fees from being directly added as if they were the depositor's contribution. It does not, however, prevent the depositor from receiving ownership over those assets.

For an initialized vault, newly minted shares are calculated exclusively from position liquidity:

```
shares = (supply * liquidityAdded) / liqBefore;
```

Neither pending fees nor retained balances appear in the denominator. Consequently, a depositor pays only for their share of `L`, but receives the same percentage claim over all vault-owned assets that will later be converted into `L`.

An attacker can exploit this asymmetry as follows:

1. Wait until existing holders have earned pending fees below the auto-compound threshold.
2. Make a large balanced deposit while `_compound()` is skipped.
3. `_onDeposit()` realizes the historical fees and retains them at the vault.
4. The attacker receives most of the share supply without paying for those retained fees.
5. Increase claimable value above the threshold, for example by supplying a relatively small additional balance.
6. Trigger another deposit, causing the retained assets to be compounded before any new shares are minted.
7. The attacker's existing shares receive most of the resulting `L` increase, even though the underlying historical fees were earned before the attacker entered.

This is the deposit-side mirror of the redemption asymmetry. On redemption, an exiting holder forfeits their share of assets outside `L`; on deposit, an entering holder acquires a share of those assets without paying for them. Both behaviors have the same root cause:

Share minting and burning account **for** position liquidity `L`,
but not the vault's complete asset inventory.

Location of Affected Code

File: [contracts/VltUsdcVault.sol:410-438](#)

```
[, uint256 claimableUsdc,) = compoundClaimable();

if (claimableUsdc >= AUTO_COMPOUND_MIN_USDC && positionLiquidity() > 0) {
    _compound();
}

uint256 retain0 = _selfBalance(currency0);
uint256 retain1 = _selfBalance(currency1);

vlt.safeTransferFrom(msg.sender, address(this), vltAmount);
usdc.safeTransferFrom(msg.sender, address(this), usdcAmount);

uint128 liqBefore = positionLiquidity();
```

File: [contracts/VltUsdcVault.sol:452-479](#)

```
(uint128 liquidityAdded, uint256 paid0, uint256 paid1) =
    abi.decode(res, (uint128, uint256, uint256));

require(liquidityAdded > 0, "no-liquidity-added");

uint256 supply = totalSupply();
if (supply == 0) {
```

```

require(liquidityAdded > MINIMUM_LIQUIDITY, "below-min-liquidity");
_mint(address(Oxdead), MINIMUM_LIQUIDITY);
shares = liquidityAdded - MINIMUM_LIQUIDITY;
} else {
    shares = (supply * liquidityAdded) / liqBefore;
}

require(shares >= minShares, "slippage-shares");
_mint(recipient, shares);

```

File: [contracts/VltUdcVault.sol:668-715](#)

```

if (positionLiquidity() > 0) {
    [ BalanceDelta fees] = poolManager.modifyLiquidity(
        poolKey,
        ModifyLiquidityParams({
            tickLower: tickLower,
            tickUpper: tickUpper,
            liquidityDelta: 0,
            salt: bytes32(0)
        }),
        ""
    );

    if (f0 > 0) {
        poolManager.take(currency0, address(this), uint256(uint128(f0)));
        retain0 += uint256(uint128(f0));
    }
    if (f1 > 0) {
        poolManager.take(currency1, address(this), uint256(uint128(f1)));
        retain1 += uint256(uint128(f1));
    }
}

(uint128 liquidityAdded, uint256 paid0, uint256 paid1) = _addLiquidity(
    bal0 > retain0 ? bal0 - retain0 : 0,
    bal1 > retain1 ? bal1 - retain1 : 0
);

```

Uniswap v4 confirms that `feesAccrued` is calculated and folded into `callerDelta` whenever a position is modified:

File: [v4-core/src/PoolManager.sol:158-183](#)

```

(principalDelta, feesAccrued) = pool.modifyLiquidity(...);

// fee delta and principal delta are both accrued to the caller
callerDelta = principalDelta + feesAccrued;

```

```
_accountPoolBalanceDelta(key, callerDelta, msg.sender);
```

Impact

A large depositor can acquire most of the economic ownership of fees and retained balances accumulated before entry without paying for them. Existing holders lose the corresponding portion of historical yield when those assets are later compounded.

Under an unmanipulated spot price, the simplest pending-fee version is bounded by the \$100 auto-compound threshold, which limits individual attack value. The same accounting defect also applies to retained balances and should be fixed together with the redemption-side forfeiture issue. The bounded value, substantial capital requirement, and limited profitability support Low severity.

Proof of Concept

The runnable PoC is located at:

```
test/audit/additional-claims.test.js
```

Run:

```
yarn hardhat test test/audit/additional-claims.test.js
```

Relevant test:

```
it("mints a new depositor claims over fees earned before entry when the gate is closed", async () => {
  const ctx = await loadFixture(deployVaultFixture);

  await [await deposit(ctx, ctx.alice, USDC(100_000))].wait();
  await removeBaseLiquidity(ctx);

  // Existing holders earn approximately 50 USDC before Bob enters.
  await [await swapExact(ctx, ctx.seeder, ctx.usdcUsdCurrency0, USDC(5_000))].wait();
  const [, claimableBeforeBob] = await ctx.vault.compoundClaimable();
  expect(claimableBeforeBob).toBeGreaterThan(0n);
  expect(claimableBeforeBob).toBeLessThan(
    await ctx.vault.AUTO_COMPOUND_MIN_USDC()
  );

  // Bob enters after the fees were earned. No compound occurs.
  const bobReceipt = await [
    await deposit(ctx, ctx.bob, USDC(1_000_000))
  ].wait();
  expect(events(ctx.vault, bobReceipt, "Compound")).toHaveLength(0);
});
```

```

const retained = events[ctx.vault, bobReceipt, "FeesRetained"][0];
expect(retained.usdcFees).to.be.greaterThan(0n);

const bobShares = await ctx.vault.balanceOf(ctx.bob.address);
const supplyBeforeCompound = await ctx.vault.totalSupply();
expect(bobShares * 100n / supplyBeforeCompound).to.be.greaterThan(85n);

// Open the compound gate and trigger it through a third-party deposit
await (await ctx.vlt.mint(ctx.alice.address, VLT(50))).wait();
await (await ctx.usdc.mint(ctx.alice.address, USDC(100))).wait();
await (await ctx.vlt.connect(ctx.alice).transfer(ctx.vault.target, VLT(50))).wait();
await (await ctx.usdc.connect(ctx.alice).transfer(ctx.vault.target, USDC(100))).wait();

const triggerReceipt = await [
  await deposit(ctx, ctx.carol, USDC(1_000))
].wait();
const compoundEvent = events[ctx.vault, triggerReceipt, "Compound"][0];

expect(compoundEvent.liquidityAdded).to.be.greaterThan(0n);

const bobCompoundClaim =
  compoundEvent.liquidityAdded * bobShares / supplyBeforeCompound;

expect(bobCompoundClaim).to.be.greaterThan(
  compoundEvent.liquidityAdded * 85n / 100n
);
});

```

Observed output:

```

claimableBeforeBob: 49.999999 USDC
retainedHistoricalUsdc: 49.999999 USDC
bobOwnershipBps: 9050
compoundLiquidity: 74210856685528
bobCompoundLiquidityClaim: 67162185334656

```

Bob owned no shares when the historical fees were generated, but obtained approximately 90.5% of the liquidity created when those assets were compounded.

Recommendation

Use a share-pricing model that includes every vault-owned asset, not only position liquidity. Mint and burn calculations should account for:

```

position principal
+ pending position fees
+ retained token balances
+ compound dust

```

Team Response

Acknowledged.

FINDING I-02

INFO

Subsequent Deposits Can Add Liquidity While Minting Zero Shares

FIXED

Description

For an initialized vault, `deposit()` calculates shares with floor division:

```
shares = (supply * liquidityAdded) / liqBefore;
```

Auto-compounding increases the position liquidity without minting shares. Therefore, after any successful compound, `liqBefore` can exceed `supply`. A sufficiently small but positive liquidity addition then satisfies:

```
supply * liquidityAdded < liqBefore
```

and produces `shares == 0`.

The contract verifies only that `liquidityAdded > 0`. It does not verify that the resulting share amount is also positive. If the depositor supplies `minShares == 0`, the slippage check accepts the zero result and `_mint(recipient, 0)` completes successfully. The depositor's consumed tokens remain in the position and increase its liquidity, but the depositor receives no ownership claim.

The first-deposit branch is not affected. It requires `liquidityAdded > MINIMUM_LIQUIDITY`, so `liquidityAdded - MINIMUM_LIQUIDITY` is always positive.

Uniswap V2 explicitly protects the equivalent post-initialization rounding case by checking the calculated liquidity after both minting branches. In [UniswapV2Pair.sol:119-126](#), it rejects a mint when integer division produces zero liquidity:

```
if (_totalSupply == 0) {
    liquidity = Math.sqrt(amount0.mul(amount1)).sub(MINIMUM_LIQUIDITY);
    _mint(address(0), MINIMUM_LIQUIDITY);
} else {
    liquidity = Math.min(
        amount0.mul(_totalSupply) / _reserve0,
        amount1.mul(_totalSupply) / _reserve1
    );
}
require(liquidity > 0, "UniswapV2: INSUFFICIENT_LIQUIDITY_MINTED");
```

Location of Affected Code

File: [contracts/VltUsdcVault.sol:448-475](#)

```
[uint128 liquidityAdded, uint256 paid0, uint256 paid1] =
  abi.decode(res, [uint128, uint256, uint256]);

require(liquidityAdded > 0, "no-liquidity-added");

uint256 supply = totalSupply();
if (supply == 0) {
  require(liquidityAdded > MINIMUM_LIQUIDITY, "below-min-liquidity");
  _mint(address(Oxdead), MINIMUM_LIQUIDITY);
  shares = liquidityAdded - MINIMUM_LIQUIDITY;
} else {
  shares = (supply * liquidityAdded) / liqBefore;
}

require(shares >= minShares, "slippage-shares");
_mint(recipient, shares);
```

Impact

A subsequent depositor who uses `minShares == 0` can lose the tokens consumed by a positive-liquidity deposit while receiving zero shares. Impact is Low because the caller controls `minShares` and can prevent the loss by requiring at least one share.

Proof of Concept

Add the following test as `test/audit.zero-share-deposit.test.js` and run:

```
yarn hardhat test test/audit.zero-share-deposit.test.js
```

```
const { expect } = require("chai");
const { ethers } = require("hardhat");
const { loadFixture } = require("@nomicfoundation/hardhat-network-helpers");
const { deployVaultFixture, deposit } = require("../helpers/setup");

const USDC = (n) => BigInt(n) * 10n ** 6n;
const VLT = (n) => BigInt(n) * 10n ** 18n;

async function eventArgs(contract, receipt, name) {
  for (const log of receipt.logs) {
    try {
      const parsed = contract.interface.parseLog(log);
      if (parsed?.name === name) return parsed.args;
    } catch {}
  }
}
```

```

}
return null;
}

describe("Zero-share deposit verification", () => {
  it("accepts positive liquidity while minting zero shares after initialization", async () => {
    const ctx = await loadFixture(deployVaultFixture);

    await (await deposit(ctx, ctx.alice, USDC(10_000))).wait();

    // Anyone can donate tokens. The next deposit auto-compounds them without
    // minting shares, producing positionLiquidity > totalSupply.
    await (await ctx.vlt.mint(ctx.alice.address, VLT(50))).wait();
    await (await ctx.usdc.mint(ctx.alice.address, USDC(100))).wait();
    await (await ctx.vlt.connect(ctx.alice).transfer(ctx.vault.target, VLT(50))).wait();
    await (await ctx.usdc.connect(ctx.alice).transfer(ctx.vault.target, USDC(100))).wait();

    const vltAmount = 707_107n;
    await (await ctx.vlt.mint(ctx.bob.address, vltAmount)).wait();
    await (await ctx.usdc.mint(ctx.bob.address, 1n)).wait();
    await (await ctx.vlt.connect(ctx.bob).approve(ctx.vault.target, ethers.MaxUint256)).wait();
    await (await ctx.usdc.connect(ctx.bob).approve(ctx.vault.target, ethers.MaxUint256)).wait();

    const supplyBefore = await ctx.vault.totalSupply();
    const liquidityBefore = await ctx.vault.positionLiquidity();
    const bobVltBefore = await ctx.vlt.balanceOf(ctx.bob.address);
    const bobUsdcBefore = await ctx.usdc.balanceOf(ctx.bob.address);

    const receipt = await (
      await ctx.vault
        .connect(ctx.bob)
        .deposit(vltAmount, 1n, 0n, ethers.MaxUint256, ctx.bob.address)
    ).wait();
    const depositEvent = await eventArgs(ctx.vault, receipt, "Deposit");
    const compoundEvent = await eventArgs(ctx.vault, receipt, "Compound");

    const supplyAfter = await ctx.vault.totalSupply();
    const liquidityAfter = await ctx.vault.positionLiquidity();
    const bobVltSpent = bobVltBefore - (await ctx.vlt.balanceOf(ctx.bob.address));
    const bobUsdcSpent = bobUsdcBefore - (await ctx.usdc.balanceOf(ctx.bob.address));

    expect(depositEvent.sharesOut).toEqual(0n);
    expect(depositEvent.liquidityAdded).toEqual(1n);
    expect(await ctx.vault.balanceOf(ctx.bob.address)).toEqual(0n);
    expect(supplyAfter).toEqual(supplyBefore);
    expect(compoundEvent).toEqual(null);
    expect(liquidityAfter).toEqual(
      liquidityBefore + compoundEvent.liquidityAdded + depositEvent.liquidityAdded
    );
  });
}

```

```

const depositLiquidityBefore = liquidityAfter - depositEvent.liquidityAdded;
expect(supplyBefore * depositEvent.liquidityAdded).toBe<lessThan>(depositLiquidityBefore);
expect(bobVltSpent).to<equal>(depositEvent.vltUsed);
expect(bobUsdcSpent).to<equal>(depositEvent.usdcUsed);
expect(bobVltSpent + bobUsdcSpent).toBe<greaterThan>(0n);

console.log({
  vltAmount: vltAmount<toString>(),
  liquidityAdded: depositEvent.liquidityAdded<toString>(),
  sharesMinted: depositEvent.sharesOut<toString>(),
  vltSpent: bobVltSpent<toString>(),
  usdcSpent: bobUsdcSpent<toString>(),
});
});
});

```

Observed output:

```

vltAmount: 707107
liquidityAdded: 1
sharesMinted: 0
vltSpent: 707107
usdcSpent: 1
1 passing

```

The public deposit adds one unit of liquidity and consumes both supplied token amounts, while Bob's share balance and total supply remain unchanged.

Recommendation

Require a positive share result after both share-calculation branches:

```

require(shares > 0, "zero-shares-minted");
require(shares >= minShares, "slippage-shares");

```

Team Response

Fixed.

Redeemers Forfeit Their Pro-Rata Share of Uncompounded Fees and Retained Balances

ACKNOWLEDGED

Description

Vault shares are denominated only in the Uniswap v4 position's raw liquidity `L`. However, holder-owned value can exist outside `L` in two forms:

- pending pool fees that have accrued to the position but have not yet been compounded; and
- token balances retained by the vault from earlier fee harvests and compound dust.

`redeem()` calculates the liquidity removed solely from the current position liquidity:

```
uint256 supply = totalSupply();
uint128 liq = positionLiquidity();
uint128 liquidityToRemove = uint128((uint256(liq) * shares) / supply);
```

When `_onRedeem()` calls `modifyLiquidity()` with a negative liquidity delta, Uniswap v4 realizes the fees accrued by the entire position. This is because `Position.update()` calculates fees using the position's full pre-modification liquidity before applying `liquidityDelta`:

```
uint128 liquidity = self.liquidity;

if (liquidityDelta == 0) {
    if (liquidity == 0) CannotUpdateEmptyPosition.selector revertWith();
} else {
    self.liquidity = LiquidityMath.addDelta(liquidity, liquidityDelta);
}

feesOwed0 = FullMath.mulDiv(
    feeGrowthInside0X128 - self.feeGrowthInside0LastX128,
    liquidity,
    FixedPoint128.Q128
);
```

The PoolManager returns `callerDelta = principalDelta + feesAccrued`. The vault correctly separates these values to prevent a partial redeemer from taking all position fees, but it transfers only principal to the redeemer and sends the complete realized fee amount to the vault:

```
BalanceDelta principalDelta = callerDelta - feesAccrued;

if (amount0 > 0) poolManager.take(currency0, d.receiver, amount0);
if (amount1 > 0) poolManager.take(currency1, d.receiver, amount1);
```

```
if (f0 > 0) poolManager.take(currency0, address(this), uint256(uint128(f0)));
if (f1 > 0) poolManager.take(currency1, address(this), uint256(uint128(f1)));
```

Pre-existing retained balances are also not included in the redemption. Once the caller's shares are burned, the caller no longer participates in a later compound. Consequently, an exiting holder forfeits approximately:

$$(\text{pending fees} + \text{retained balances}) \times \text{shares redeemed} / \text{total supply}$$

The value remains in the vault and benefits the remaining holders when a later deposit triggers compounding. If all live holders exit, the value becomes economically associated with the permanently locked minimum-liquidity shares and may remain orphaned.

`previewRedeem()` mirrors the principal-only implementation and therefore accurately predicts the current redemption output, but it also excludes the exiting holder's fee entitlement.

The issue is Low severity because only accrued yield and retained balances are affected, no external attacker can force a holder to redeem, and a holder can avoid a material forfeiture by triggering compounding before exiting when the vault's `claimableUsdc` is at least `AUTO_COMPOUND_MIN_USDC`. When the gate is open, any valid nonzero two-token deposit can trigger compounding; the deposit itself does not need to be worth \$100.

Location of Affected Code

File: [contracts/VltUsdcVault.sol:332-353](#)

```
// @notice Two-token analog of ERC-4626 'previewRedeem': the principal (VLT, USDC) a
// redeemer would receive for 'shares' at the CURRENT price, WITHOUT executing. Excludes the
// position's uncompounded fees (those are retained for all holders, exactly as redeem() does).
function previewRedeem(uint256 shares) public view returns (uint256 vltAmount, uint256 usdcAmount) {
    uint256 supply = totalSupply();
    if (shares == 0 || supply == 0) return (0, 0);
    uint128 liquidityToRemove = uint128((uint256(positionLiquidity()) * shares) / supply);
    // ... calculates principal represented by liquidityToRemove only
}
```

File: [contracts/VltUsdcVault.sol:503-521](#)

```
uint256 supply = totalSupply();
uint128 liq = positionLiquidity();
uint128 liquidityToRemove = uint128((uint256(liq) * shares) / supply);

_burn(msg.sender, shares);

bytes memory res = poolManager.unlock(
```

```
abi.encode(Action.REDEEM, abi.encode(RedeemData(receiver, liquidityToRemove)))
];
```

File: [contracts/VltUsdcVault.sol:718-749](#)

```
(BalanceDelta callerDelta, BalanceDelta feesAccrued) = poolManager.modifyLiquidity(
    poolKey,
    ModifyLiquidityParams({
        tickLower: tickLower,
        tickUpper: tickUpper,
        liquidityDelta: -int256(uint256(d.liquidity)),
        salt: bytes32(0)
    }),
    ""
);
```

```
BalanceDelta principalDelta = callerDelta - feesAccrued;
```

```
if (amount0 > 0) poolManager.take(currency0, d.receiver, amount0);
```

```
if (amount1 > 0) poolManager.take(currency1, d.receiver, amount1);
```

```
if (f0 > 0) poolManager.take(currency0, address(this), uint256(uint128(f0)));
```

```
if (f1 > 0) poolManager.take(currency1, address(this), uint256(uint128(f1)));
```

File: [v4-core/src/libraries/Position.sol:82-96](#)

```
uint128 liquidity = self.liquidity;
```

```
if (liquidityDelta == 0) {
```

```
    if (liquidity == 0) CannotUpdateEmptyPosition.selector.revertWith();
```

```
} else {
```

```
    self.liquidity = LiquidityMath.addDelta(liquidity, liquidityDelta);
```

```
}
```

```
unchecked {
```

```
    feesOwed0 =
```

```
        FullMath.mulDiv(feeGrowthInside0X128 - self.feeGrowthInside0LastX128, liquidity, FixedPoint128.Q128);
```

```
    feesOwed1 =
```

```
        FullMath.mulDiv(feeGrowthInside1X128 - self.feeGrowthInside1LastX128, liquidity, FixedPoint128.Q128);
```

```
}
```

Impact

An existing holder loses their pro-rata share of pending fees and retained vault balances to the remaining holders. Principal remains safe, and the loss is voluntary and avoidable when compounding can be triggered, limiting the issue to Low severity.

Proof of Concept

Add the following test as `test/audit.asym001.test.js` and run:

```
yarn hardhat test test/audit.asym001.test.js
```

```
const { expect } = require("chai");
const { loadFixture } = require("@nomicfoundation/hardhat-network-helpers");
const {
  deployVaultFixture,
  deposit,
  redeem,
  swapExact,
  removeBaseLiquidity,
} = require("../helpers/setup");

const USDC = (n) => BigInt(n) * 10n ** 6n;
const VLT = (n) => BigInt(n) * 10n ** 18n;

async function eventArgs(contract, receipt, name) {
  for (const log of receipt.logs) {
    try {
      const parsed = contract.interface.parseLog(log);
      if (parsed?.name === name) return parsed.args;
    } catch {}
  }
  return null;
}

describe("ASYM-001 verification", () => {
  it("an exiting holder receives principal only and forfeits pro-rata pending/retained fees", async () => {
    const ctx = await loadFixture(deployVaultFixture);

    await (await deposit(ctx, ctx.alice, USDC(50_000))).wait();
    await (await deposit(ctx, ctx.bob, USDC(50_000))).wait();
    await removeBaseLiquidity(ctx);

    // Generate approximately balanced fees while the vault is the sole LP.
    for (let i = 0; i < 10; i++) {
      await (await swapExact(ctx, ctx.seeder, ctx.usdcIsCurrency0, USDC(1_000))).wait();
      await (await swapExact(ctx, ctx.seeder, !ctx.usdcIsCurrency0, VLT(500))).wait();
    }

    const aliceShares = await ctx.vault.balanceOf(ctx.alice.address);
    const bobShares = await ctx.vault.balanceOf(ctx.bob.address);
    const supplyBefore = await ctx.vault.totalSupply();
    const [pendingVlt, pendingUsdc, claimableValue] = await ctx.vault.compoundClaimable();

    expect(pendingVlt).toBeGreaterThan(0n);
  });
});
```

```

expect(pendingUsdc).to.be.greaterThan(0n);
expect(claimableValue).to.be.greaterThanOrEqual(await ctx.vault.AUTO_COMPOUND_MIN_USDC());

const aliceFeeClaimVlt = (pendingVlt * aliceShares) / supplyBefore;
const aliceFeeClaimUsdc = (pendingUsdc * aliceShares) / supplyBefore;
expect(aliceFeeClaimVlt).to.be.greaterThan(0n);
expect(aliceFeeClaimUsdc).to.be.greaterThan(0n);

const [previewVlt, previewUsdc] = await ctx.vault.previewRedeem(aliceShares);
const aliceVltBefore = await ctx.vlt.balanceOf(ctx.alice.address);
const aliceUsdcBefore = await ctx.usdc.balanceOf(ctx.alice.address);

const redeemReceipt = await (await redeem(ctx, ctx.alice, aliceShares)).wait();
const redeemEvent = await eventArgs(ctx.vault, redeemReceipt, "Redeem");
const retainedEvent = await eventArgs(ctx.vault, redeemReceipt, "FeesRetained");

const aliceVltReceived = (await ctx.vlt.balanceOf(ctx.alice.address)) - aliceVltBefore;
const aliceUsdcReceived = (await ctx.usdc.balanceOf(ctx.alice.address)) - aliceUsdcBefore;

// Alice receives only the principal-only preview, while all fees remain at the vault
expect(aliceVltReceived).to.equal(redeemEvent.vltOut);
expect(aliceUsdcReceived).to.equal(redeemEvent.usdcOut);
expect(aliceVltReceived > previewVlt ? aliceVltReceived - previewVlt : previewVlt - aliceVltReceived)
.to.be.lessThanOrEqual(2n);
expect(aliceUsdcReceived > previewUsdc ? aliceUsdcReceived - previewUsdc : previewUsdc - aliceUsdcReceived)
.to.be.lessThanOrEqual(2n);
expect(await ctx.vlt.balanceOf(ctx.vault.target)).to.equal(retainedEvent.vltFees);
expect(await ctx.usdc.balanceOf(ctx.vault.target)).to.equal(retainedEvent.usdcFees);

// A later deposit compounds the retained value for remaining holders before
// the new depositor's shares are minted.
const bobClaimBefore = ((await ctx.vault.positionLiquidity()) * bobShares) /
  (await ctx.vault.totalSupply());
const depositReceipt = await (await deposit(ctx, ctx.carol, USDC(1_000))).wait();
const compoundEvent = await eventArgs(ctx.vault, depositReceipt, "Compound");
expect(compoundEvent).to.not.equal(null);
expect(compoundEvent.liquidityAdded).to.be.greaterThan(0n);
const bobClaimAfter = ((await ctx.vault.positionLiquidity()) * bobShares) /
  (await ctx.vault.totalSupply());
expect(bobClaimAfter).to.be.greaterThan(bobClaimBefore);

console.log("alice fee claim forfeited {raw}:", {
  vlt: aliceFeeClaimVlt.toString(),
  usdc: aliceFeeClaimUsdc.toString(),
  retainedVlt: retainedEvent.vltFees.toString(),
  retainedUsdc: retainedEvent.usdcFees.toString(),
  bobLiquidityClaimIncrease: (bobClaimAfter - bobClaimBefore).toString(),
});
});
});
});

```

Observed output:

```
alice fee claim forfeited (raw): {  
  vlt: '2499999999999999292892',  
  usdc: '49999999',  
  retainedVlt: '4999999999999999999',  
  retainedUsdc: '99999999',  
  bobLiquidityClaimIncrease: '70647833232368'  
}  
1 passing
```

Alice owned approximately half the shares. She forfeited approximately 25 VLT and 50 USDC of accrued fees, while the full 50 VLT and 100 USDC fee balances stayed in the vault. A later compound increased Bob's liquidity claim, demonstrating the redistribution to the remaining holder.

Recommendation

Include holder-owned pending fees and retained balances in redemption accounting. One approach is to compound before calculating `liquidityToRemove` whenever the position exists:

```
if (positionLiquidity() > 0) {  
  _compound();  
}  
  
uint256 supply = totalSupply();  
uint128 liq = positionLiquidity();  
uint128 liquidityToRemove = uint128((uint256(liq) * shares) / supply);
```

Because compounding can leave unbalanced token dust outside `L`, the most complete fix is to calculate the redeemer's ratio using the pre-burn supply, send that ratio of `feesAccrued` to the receiver, and transfer the same ratio of pre-existing retained balances.

The remaining fees and balances should stay at the vault for the remaining shares. Alternatively, restore a permissionless external compounding function so holders can fold this value into `L` before exiting. Update `previewRedeem()` to apply the same accounting model as `redeem()`.

Team Response

Acknowledged.

Donation-Inflated Fresh Fees Allow Profitable Sandwiches of the Compound Rebalance

ACKNOWLEDGED

Description

Whenever `compoundClaimable()` reaches `AUTO_COMPOUND_MIN_USDC`, the next deposit automatically harvests pending fees, rebalances the vault's token inventory through the same VLT/USDC pool, and adds the resulting balances as liquidity.

The rebalance uses the pool's instantaneous `sqrtPriceX96` for all safety-critical decisions:

- determining which currency is overweight;
- converting one currency into the other for imbalance measurement;
- selecting `amountIn`; and
- constructing `sqrtPriceLimitX96`.

```
[uint160 sqrtPriceX96,...] = poolManager.getSlotO(poolKey.told());
uint256 sp = uint256(sqrtPriceX96);

uint256 valueOfInCcy1 = FullMath.mulDiv(FullMath.mulDiv(r0, sp, Q96), sp, Q96);
```

The price limit is derived from that same current spot price:

```
uint256 limit = zeroForOne
? FullMath.mulDiv(sp, SQRT_LIMIT_DOWN_BPS, BPS)
: FullMath.mulDiv(sp, SQRT_LIMIT_UP_BPS, BPS);
```

This bounds the vault's own price impact, but it does not bound execution against an external reference price. The contract itself documents this limitation (`contracts/VltUsdcVault.sol:97-101`): the price limit "bounds the price impact OF THE VAULT'S OWN SWAP, not a prior manipulation — the filled portion executes at the prevailing (possibly skewed) spot price." If an attacker first manipulates spot, the permitted execution range is simply:

manipulated spot × permitted movement

The vault also performs no minimum-output check. `_swapExactIn()` relies solely on the price limit as its slippage guard:

```
BalanceDelta delta = poolManager.swap(
    poolKey,
    SwapParams{
```

```

zeroForOne: zeroForOne,
amountSpecified: -int256(amountIn),
sqrtPriceLimitX96: sqrtPriceLimitX96
}},
};

```

Why imbalance-based direction does not prevent a sandwich

A prior audit hypothesis concluded that `_rebalance()` always trades against manipulation because manipulation changes which token appears overweight. That is only true when the vault starts with sufficiently balanced holdings.

When pending or retained assets are strongly one-sided, the balance imbalance fixes the swap direction. For example, with `r0 > 0` and `r1 == 0`:

```

if (r1 > value0InCcy1) {
  zeroForOne = false;
  amountIn = (r1 - value0InCcy1) / 2;
} else {
  uint256 value1InCcy0 = FullMath.mulDiv(FullMath.mulDiv(r1, Q96, sp), Q96, sp);
  zeroForOne = true;
  amountIn = (r0 > value1InCcy0 ? r0 - value1InCcy0 : 0) / 2;
}

```

Since `r1 == 0`, the condition remains false for every nonzero spot price. The vault therefore remains committed to `zeroForOne`, including after an attacker front-runs with another `zeroForOne` swap.

The attack becomes a conventional same-direction sandwich:

1. A sufficiently large one-sided `donate()` credit increases the vault position's pending `feesAccrued` without moving spot.
2. Attacker swaps `currency0` for `currency1`, pushing spot against the vault.
3. Attacker triggers `deposit()`, which enters `_compound()` before minting deposit shares.
4. The vault harvests one-sided fees and swaps `currency0` for `currency1` in the same direction as the attacker.
5. The vault's swap pushes the price further and executes relative to the already-manipulated spot.
6. Attacker swaps the acquired `currency1` back to `currency0`.
7. Attacker realizes positive round-trip profit while the vault suffers an equivalent reference-price loss.

The reviewed implementation has no fresh-fee cap

At the review commit, `_rebalance(r0, r1)` sizes the swap from the vault's full loose balances. It directly selects approximately half of the spot-valued imbalance, subject only to the current-spot-relative 5% price-movement limit:

```

if (r1 > value0InCcy) {
    zeroForOne = false;
    amountIn = (r1 - value0InCcy) / 2;
} else {
    uint256 value1InCcy0 = FullMath.mulDiv(FullMath.mulDiv(r1, Q96, sp), Q96, sp);
    zeroForOne = true;
    amountIn = (r0 > value1InCcy0 ? r0 - value1InCcy0 : 0) / 2;
}

```

Therefore, a large one-sided donation materialized as `feesAccrued` creates both the trigger and the one-sided loose balance used to size the victim swap. The absence of the removed fresh-fee cap does not eliminate the demonstrated sandwich mechanism; it means the exposure is determined directly by the donated balance and the pool-price limit.

Uniswap v4 explicitly warns that `feesAccrued` can be artificially inflated through donations (`v4-core/src/interfaces/IPoolManager.sol:130`):

```

/// @dev Note that feesAccrued can be artificially inflated by a malicious actor and integrators should be careful using the
value
function modifyLiquidity(PoolKey memory key, ModifyLiquidityParams memory params, bytes callData hookData)
    external
    returns (BalanceDelta callerDelta, BalanceDelta feesAccrued);

```

The PoC uses the real v4 `donate()` path to construct the one-sided fee state without directly transferring tokens into the vault or mocking accounting. The sandwich attacker is separate from the donor and does not pay the donation cost. Self-funding the donation is not the demonstrated attack: combining the \$100,000 donor cost with the \$3,929.58 sandwich profit leaves that actor approximately \$96,070.42 negative before gas.

Profitability depends on balance composition, donation size, liquidity depth, and manipulation amount. The finding does not claim every compound, or every one-sided state created by ordinary swap volume, is profitably sandwichable. It demonstrates a donation-created state where the contract's stated protection fails and an unprivileged attacker extracts vault value. The official pitch mentions ecosystem fee routing at `docs/vltUSDC-pitch.md:17,45`, but it does not specify that routing will use `PoolManager.donate()`; operational likelihood therefore depends on how that routing is implemented.

Location of Affected Code

File: [contracts/VltUsdcVault.sol](#)

```

[, uint256 claimableUsdc] = compoundClaimable();
if (claimableUsdc >= AUTO_COMPOUND_MIN_USDC && positionLiquidity() > 0) {
    _compound();
}

```

File: [contracts/VltUsdcVault.sol](#)

```
_rebalance(_selfBalance(currency0), _selfBalance(currency1));  
// ...  
[uint128 liquidityAdded,,] = _addLiquidity(_selfBalance(currency0), _selfBalance(currency1));
```

File: [contracts/VltUsdcVault.sol](#)

```
function _rebalance(uint256 r0, uint256 r1) internal {  
    [uint160 sqrtPriceX96,,] = poolManager.getSlot0(poolKey.told());  
    uint256 sp = uint256(sqrtPriceX96);  
  
    uint256 value0InCcy1 = FullMath.mulDiv(FullMath.mulDiv(r0, sp, Q96), sp, Q96);  
  
    bool zeroForOne;  
    uint256 amountIn;  
    if (r1 > value0InCcy1) {  
        zeroForOne = false;  
        amountIn = (r1 - value0InCcy1) / 2;  
    } else {  
        uint256 value1InCcy0 = FullMath.mulDiv(FullMath.mulDiv(r1, Q96, sp), Q96, sp);  
        zeroForOne = true;  
        amountIn = (r0 > value1InCcy0 ? r0 - value1InCcy0 : 0) / 2;  
    }  
  
    if (amountIn == 0) return;  
  
    uint256 limit = zeroForOne  
        ? FullMath.mulDiv(sp, SQRT_LIMIT_DOWN_BPS, BPS)  
        : FullMath.mulDiv(sp, SQRT_LIMIT_UP_BPS, BPS);  
    // ... clamp limit to the valid sqrt band ...  
    _swapExactIn(zeroForOne, amountIn, uint160(limit));  
}
```

File: [contracts/VltUsdcVault.sol](#)

```
function _swapExactIn(bool zeroForOne, uint256 amountIn, uint160 sqrtPriceLimitX96)  
    internal  
    returns (uint256 amountOut)  
{  
    BalanceDelta delta = poolManager.swap(  
        poolKey,  
        SwapParams({  
            zeroForOne: zeroForOne,  
            amountSpecified: -int256(amountIn),  
            sqrtPriceLimitX96: sqrtPriceLimitX96  
        }),  
        ""  
    );  
}
```

```
// settles input consumed and takes output to self — no minAmountOut check
}
```

Impact

An unprivileged attacker can extract value from the vault when a deposit-triggered compound encounters a sufficiently large one-sided donation credited as fresh fees. In the tested approximately \$2.1 million vault state, a \$120,000 round-trip manipulation produced \$3,929.58 attacker profit and a matching \$3,929.58 vault loss, after both 1% pool fees. The swap, attacker-controlled trigger deposit, and reverse swap can be ordered by the attacker; no vault shares or privileged role are required.

Loss scales with the donation credited to the vault, pool depth, and available manipulation capital. The demonstrated loss exceeds 0.01% of vault value, but the attacker cannot economically create the prerequisite themselves. An unrelated actor or protocol mechanism must first make a sufficiently large one-sided v4 donation, no in-scope component performs such donations, and the official pitch does not establish `donate()`-based fee routing as an expected operation. The issue is therefore Low Risk under the currently evidenced deployment model. It should be reconsidered as Medium only if the team confirms that large one-sided `PoolManager.donate()` calls are part of routine protocol operation.

Proof of Concept

The runnable PoC is located at `test/audit.compound-sandwich-probe.test.js`. The test harness imports the real v4 `PoolDonateTest` router through `contracts/test/V4Harness.sol`.

Run:

```
yarn hardhat test test/audit.compound-sandwich-probe.test.js
```

Core PoC sequence:

```
it("scans a same-direction sandwich around a one-sided-fee compound", async () => {
  const ctx = await loadFixture(usdcFixture);
  expect(ctx.usdc.currency).to.equal(true);

  // Approximately $2 million of vault principal, then remove the external LP so
  // fee attribution and vault value changes are deterministic.
  await [await deposit(ctx, ctx.alice, USDC(1_000_000))].wait();
  await removeBaseLiquidity(ctx);

  // Produce one-sided pending USDC fees through the real v4 donate() path.
  await [await ctx.usdc.mint(ctx.seeder.address, USDC(100_000))].wait();
  await [await ctx.usdc.connect(ctx.seeder).approve(ctx.donateRouter.target, ethers.MaxUint256)].wait();
  await [await ctx.donateRouter.connect(ctx.seeder).donate(ctx.poolKey, USDC(100_000), On, "0x")].wait();
});
```

```

const [pendingVlt, pendingUsdc, claimable] = await ctx.vault.compoundClaimable();
expect(pendingVlt).to.equal(0n);
expect(pendingUsdc).to.be.greaterThanOrEqual(USDC(99_999));
expect(claimable).to.be.greaterThan(await ctx.vault.AUTO_COMPOUND_MIN_USDC());

const vaultValueBeforeAttack = await vaultValueAtReference(ctx);

await (await ctx.usdc.mint(ctx.bob.address, USDC(1_000_000))).wait();
await (await ctx.usdc.connect(ctx.bob).approve(ctx.swapRouter.target, ethers.MaxUint256)).wait();
await (await ctx.vlt.connect(ctx.bob).approve(ctx.swapRouter.target, ethers.MaxUint256)).wait();

const usdcBefore = await ctx.usdc.balanceOf(ctx.bob.address);
const vltBefore = await ctx.vlt.balanceOf(ctx.bob.address);

// Front-run in the same direction selected by the one-sided rebalance.
await (await swapExact(ctx, ctx.bob, true, USDC(120_000))).wait();
const vltBought = (await ctx.vlt.balanceOf(ctx.bob.address)) - vltBefore;

// Any valid deposit can trigger the internal compound.
const triggerReceipt = await (await deposit(ctx, ctx.carol, USDC(100))).wait();
const compoundEvent = eventArgs(ctx.vault, triggerReceipt, "Compound");
const depositEvent = eventArgs(ctx.vault, triggerReceipt, "Deposit");
expect(compoundEvent.liquidityAdded).to.be.greaterThan(0n);

// Back-run by selling all VLT acquired in the front-run.
await (await swapExact(ctx, ctx.bob, false, vltBought)).wait();

const usdcAfter = await ctx.usdc.balanceOf(ctx.bob.address);
const attackerPnl = usdcAfter - usdcBefore;

const vaultValueAfterAttack = await vaultValueAtReference(ctx);
const triggerContribution = referenceValue(depositEvent.vltUsed, depositEvent.usdcUsed);
const vaultLoss = vaultValueBeforeAttack + triggerContribution - vaultValueAfterAttack;

expect(attackerPnl).to.be.greaterThan(0n);
expect(vaultLoss).to.be.greaterThan(0n);
});

```

Observed scan (front-run size → attacker PnL / vault loss):

Front-run	Attacker PnL	Vault loss
\$1,000	\$31.650974	\$31.650975
\$2,500	\$79.165962	\$79.165963
\$5,000	\$158.459840	\$158.459841
\$10,000	\$317.427583	\$317.427584
\$20,000	\$636.857219	\$636.857220
\$40,000	\$1,281.494033	\$1,281.494034
\$80,000	\$2,592.394879	\$2,592.394881
\$120,000	\$3,929.582089	\$3,929.582091

The near-exact equality between attacker profit and vault loss demonstrates direct value transfer rather than an accounting-only discrepancy.

Verification boundary and controls

Independent reruns against the real vendored `PoolManager` produced the following controls:

- Without the compound victim leg, the same \$120,000 front/back round trip lost \$2,262.980713 to price impact and fees.
- The checked-in ordinary-volume control generated approximately \$5,000 of one-sided USDC fees from a \$500,000 swap, but the same sandwich lost \$1,852.738160.
- Additional scans of one-way swap volume up to \$25 million did not produce positive attacker PnL for the tested manipulation sizes because the fee accrual remained coupled to the moved spot price.
- Donation-created states became profitable in the tested depth between \$10,000 and \$25,000 of one-sided donation; a \$25,000 donation with a \$200,000 manipulation produced \$752.128621 of attacker profit.
- Leaving external full-range liquidity in the pool reduced the vault's donation share but did not eliminate profitability in the tested configurations. With `5e17` external liquidity units, the vault received approximately \$58,578.64 of the \$100,000 donation and the \$120,000 sandwich still produced \$1,501.433509 of attacker profit.

Accordingly, the verified exploit is the donation-decoupled state. The report does not claim that ordinary one-way trading fees alone are sufficient.

Recommendation

Do not perform an unconditional spot-priced swap inside a permissionless deposit path.

Preferred options:

1. **Avoid the internal swap.** Add only the balanced portion of harvested assets and retain one-sided excess until future fees provide the other currency. This preserves the vault's oracle-free design and removes the sandwichable victim swap.
2. **If the swap is retained, use a manipulation-resistant reference price.** Determine direction, maximum input, and minimum output from a sufficiently long TWAP or external oracle. Reject compounding when spot deviates beyond a narrow bound from the reference.

3. **Require an enforceable minimum output.** A price limit relative to the current spot is insufficient. Calculate `minAmountOut` from the reference price and revert if the returned `BalanceDelta` does not satisfy it.
4. **Separate compounding from deposits as defence in depth.** A dedicated keeper flow with off-chain quoting and private submission reduces public sandwich exposure, but caller-supplied slippage alone is insufficient because a malicious caller can choose permissive bounds while holder assets fund the swap.

If the internal swap is retained, the contract should enforce both a spot-within-deviation check against a TWAP/reference price and an oracle-derived `amountOut` minimum. The current-spot-relative price limit may remain as a secondary exposure limit, but it is not a sufficient primary slippage control.

Team Response

Acknowledged.

FINDING I-05

`feeApr()` **Reports All No-Mint Liquidity Growth as Fee APR, Contradicting Its Pitched Yield Source**

INFO

FIXED

Description

`feeApr()` derives its APR from the growth of `positionLiquidity() * 1e18 / totalSupply()` (L/share), anchored at `1e18` at inception and sampled into a daily ring buffer by `_snapshotFeeGrowth()` after each real compound. Its NatSpec explicitly states the metric "Reflects compounded fees only" (`contracts/VltUsdcVault.sol:554`), and the official protocol documentation (`docs/vltUSDC-pitch.md`) presents vault yield as originating from trading volume and compounded trading fees.

The L/share ratio, however, is provenance-blind: it rises on **any** position-liquidity growth that occurs without corresponding share issuance, not only on growth funded by trading fees. Concretely, the following non-trading-fee sources all inflate the reported "fee" APR:

1. **Direct token donations to the vault.** `_onCompound()` reinvests the vault's FULL token balances and mints no shares (`contracts/VltUsdcVault.sol:754-762`). Balanced VLT/USDC transferred directly to the vault is therefore folded into the position at the next compound and shows up as L/share growth. Notably, the deposit share formula was deliberately hardened against exactly this vector for share pricing — shares are minted pro-rata to ΔL from the pool, with the inline comment "basing on ΔL from the pool neutralizes direct-donation inflation" (`contracts/VltUsdcVault.sol:445-447`) — but `feeApr()` applies no equivalent provenance filter.
2. **Pool donations via `PoolManager.donate()`**. Anyone can donate tokens to the vault's in-range position (`v4-core/src/PoolManager.sol:256`, `v4-core/src/libraries/Pool.sol:466`); the donation accrues through fee-growth accounting and is harvested by the vault as `feesAccrued`, indistinguishable from organic swap fees.
3. **Redeemer forfeiture concentration.** On redeem, the position's full uncompounded fees plus the vault's retained balances stay with the vault while the redeemer's shares are burned (`contracts/VltUsdcVault.sol:708-712`; see L-03, acknowledged as intentional design). Each redemption therefore concentrates pending value onto fewer shares, so the per-share growth reported by `feeApr()` can exceed the pool's actual fee generation rate — the metric measures growth *for remaining holders*, not the position's organic fee yield.
4. **Rounding dust in the vault's favour.** Deposit share minting floors (`shares = supply * liquidityAdded / liqBefore`, `contracts/VltUsdcVault.sol:447`) and redeem liquidity removal floors (`liquidityToRemove = liq * shares / supply`, `contracts/VltUsdcVault.sol:488`), each leaving dust-level L/share increments unrelated to fees.

Vault accounting is unaffected: sources 1, 3, and 4 are the documented "folds forward, benefits remaining holders" design, and share pricing itself is donation-resistant. The issue is strictly that the metric's name, its NatSpec ("Reflects compounded fees only"), and the pitch framing attribute all of this growth to trading fees.

Location of Affected Code

File: [contracts/VltUsdcVault.sol:550-564](#)

```
/// @notice Trailing fee-growth APR in basis points over the lifetime, ~7-day, and ~30-day windows,
/// derived from the L/share daily ring. Each is annualized by the ACTUAL elapsed time between the
/// matched snapshot and now. A window returns 0 when there isn't a snapshot at least that old yet
/// (insufficient history). NOTE: cadence-dependent guidance, not a guarantee — sparse or clustered
/// compounds make short windows noisy. Reflects compounded fees only (pending fees live in
/// compoundClaimable). View-only; the 30-slot scan costs nothing off-chain.
function feeApr() external view returns (uint256 lifetimeBps, uint256 d7Bps, uint256 d30Bps) {
    uint256 supply = totalSupply();
    if (supply == 0 || inceptionTime == 0) return (0, 0, 0);
    uint256 perNow = (uint256(positionLiquidity()) * 1e18) / supply;
    lifetimeBps = _annualizedBps(1e18, inceptionTime, perNow); // baseline L/share == 1.0 (1e18) at inception
    d7Bps = _windowApr(perNow, 7 days);
```

```
d30Bps = _windowApr(perNow, 30 days);  
}
```

File: [contracts/VltUsdcVault.sol:754-762](#)

```
// 4. Reinvest the FULL vault balance — this harvest PLUS retained deposit/redeem fees and  
// prior compound dust, 100% of it, with no fee carved out for anyone. Mints NO shares  
// -> L grows against fixed supply -> NAV/share rises ...  
(uint128 liquidityAdded,) = _addLiquidity(_selfBalance(currency0), _selfBalance(currency1));
```

Impact

Off-chain, users and integrations reading `feeApr()` (or a UI built on it) as organic trading-fee performance can be misled. A third party willing to spend their own tokens could even deliberately inflate the displayed APR ahead of a marketing window by donating to the vault or pool — economically irrational for profit (the donation accrues to existing shareholders) but cheap as yield-washing for a small vault. The NatSpec claim "Reflects compounded fees only" is factually incorrect as written.

Recommendation

One of:

- Rename the metric to what it measures (e.g. `liquidityGrowthApr()` / `navPerShareGrowthApr()`) and correct the NatSpec to state that donations, retained balances, redemption forfeiture, and rounding dust are included. If ABI stability matters, keep `feeApr()` as a documented alias.
- Or maintain a provenance-aware fee index: accumulate only the liquidity added from harvested `feesAccrued` (already isolated as `fee0 / fee1` in `_onCompound()`, [contracts/VltUsdcVault.sol:736-737](#)) into a dedicated counter and derive the fee APR from that, leaving the L/share ring as a separate total-growth metric.

At minimum, fix the "Reflects compounded fees only" NatSpec line.

Team Response

Fixed.

ZapHelper External Swap Functions Lack a Reentrancy Guard

Description

`ZapHelper.zap()` and `ZapHelper.zapDeposit()` are external, permissionless entry points that make several external calls per invocation — `safeTransferFrom` on a caller-named token, an arbitrary low-level `router.call(swapData)`, a token `safeTransfer` to the recipient, a `vault.deposit()` call, and finally a full-balance `_sweep()` — yet neither function carries a reentrancy guard. The contract does not inherit `ReentrancyGuard` at all.

The helper's safety model is "custody-free": it is expected to start and end every call holding a zero token balance, so each caller is isolated. That invariant holds across separate transactions, but it is not enforced across nested calls within a single transaction. During execution, the helper temporarily holds the outer caller's tokens, and its cleanup logic reads the contract's **full** live balance rather than the current frame's own delta:

```
// zapDeposit(): USDC for the LP is the entire held balance.
uint256 usdcForLp = IERC20(usdc).balanceOf(address(this)); // ZapHelper.sol:130

// _sweep(): transfers the entire held balance of a caller-named token.
function _sweep(address token, address to) internal {
    uint256 bal = IERC20(token).balanceOf(address(this)); // ZapHelper.sol:211-213
    if (bal > 0) IERC20(token).safeTransfer(to, bal);
}
```

If any external call in the flow yields control — a callback-capable / hook-bearing token in `safeTransferFrom` / `safeTransfer`, a recipient callback, or an alternate router entry point reachable through the arbitrary `router.call(swapData)` — a nested `zap()` / `zapDeposit()` can execute while the outer frame's tokens are still live in the contract. The nested and outer frames share the same contract-level balance, and `_sweep()` / `usdcForLp` do not distinguish which frame supplied the tokens. A nested execution can therefore sweep or bank balances that belong to the outer frame, and `_execRoute()`'s balance-delta output measurement can be attributed to the wrong frame.

The Universal Router's own execution lock does not fully substitute for a helper-level guard: it only blocks reentry into `UniversalRouter.execute()` while the router is locked, not callbacks that occur before the initial router call, during token transfers, during output delivery, or through other callable router entrypoints.

No exploit is demonstrated against the intended standard USDC/VLT V2/V3 production route with standard (non-callback) ERC-20 tokens; under those assumptions, a malformed route simply reverts the transaction and cannot brick the helper or affect later users. This is a defence-in-depth / code-hygiene gap for the unrestricted raw `zap()` surface and for any future support of callback-capable tokens or hook-bearing routes.

Location of Affected Code

File: [contracts/ZapHelper.sol:98-106](#)

— `zapDeposit()` external entrypoint, no `nonReentrant`.

File: [contracts/ZapHelper.sol:151-163](#)

— `zap()` external entrypoint, no `nonReentrant`:

```
function zap(  
    address tokenIn,  
    address tokenOut,  
    uint256 amountIn,  
    uint256 minOut,  
    address recipient,  
    bytes calldata swapData  
) external returns (uint256 amountOut) {  
    IERC20(tokenIn).safeTransferFrom(msg.sender, address(this), amountIn);  
    amountOut = _execRoute(tokenIn, tokenOut, amountIn, minOut, swapData);  
    IERC20(tokenOut).safeTransfer(recipient, amountOut);  
    _sweep(tokenIn, msg.sender);  
}
```

File: [contracts/ZapHelper.sol:167-185](#)

— `_execRoute()` forwards arbitrary caller-supplied calldata to the router via a low-level call:

```
(bool ok, bytes memory ret) = router.call(swapData);
```

File: [contracts/ZapHelper.sol:130](#) and [contracts/ZapHelper.sol:211-213](#)

— full-balance reads (`usdcForLp()`, `_sweep()`) that assume the helper holds nothing from any other frame.

Impact

Under the intended standard-ERC-20 + honest-immutable-router deployment, there is no demonstrated fund loss: the flow either completes cleanly or reverts. The issue is that the contract's isolation guarantee rests on an unenforced "holds nothing between calls" assumption rather than on an explicit guard. Should the helper ever route a callback-capable token, a hook-bearing route, or a router entry point that hands back control mid-flow, the shared full-balance reads (`_sweep` , `usdcForLp`) allow one frame's tokens to be attributed to another. Adding a reentrancy guard removes the assumption entirely at negligible cost.

Recommendation

Inherit OpenZeppelin's `ReentrancyGuard` and apply `nonReentrant` to both external entrypoints:

```
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract ZapHelper is IZapHelper, ReentrancyGuard {
    function zapDeposit(...) external nonReentrant returns (uint256 shares) { ... }
    function zap(...) external nonReentrant returns (uint256 amountOut) { ... }
}
```

This is a low-cost, defensive hardening that makes the custody-free isolation property hold by construction rather than by assumption, and future-proofs the raw `zap()` surface against callback-capable tokens and non-standard router routes.

Team Response

Fixed.

FINDING I-07

`zap()` **Has No Deadline, Allowing Stale Swap Transactions to Execute Later**

● INFO

FIXED

Description

`ZapHelper.zap()` exposes the raw swap path but does not accept or enforce a deadline. Unlike `zapDeposit()` , a signed `zap()` transaction can remain executable until it is mined, as long as the caller's `minOut` still passes.

`zapDeposit()` checks expiry before executing the external route:

```
require(block.timestamp <= deadline, "expired");
```

However, `zap()` has no deadline parameter or timestamp check. This means stale `zap()` transactions can execute under changed market conditions. `minOut` limits the minimum received amount, but it does not limit when the transaction may execute.

Location of Affected Code

File: [contracts/ZapHelper.sol](#)

```
function zap(  
    address tokenIn,  
    address tokenOut,  
    uint256 amountIn,  
    uint256 minOut,  
    address recipient,  
    bytes calldata swapData  
) external returns (uint256 amountOut) {  
    IERC20(tokenIn).safeTransferFrom(msg.sender, address(this), amountIn);  
    amountOut = _execRoute(tokenIn, tokenOut, amountIn, minOut, swapData);  
    IERC20(tokenOut).safeTransfer(recipient, amountOut);  
    _sweep(tokenIn, msg.sender);  
}
```

Impact

A `zap()` caller may have an old swap executed much later than intended, exposing them to stale-route or MEV execution within their `minOut` tolerance. The loss is bounded by the caller's `minOut`, so this is a low-severity issue, but it is avoidable and inconsistent with `zapDeposit()`.

Recommendation

Add a deadline parameter to `zap()` and check it before pulling tokens or executing the route.

```
require(block.timestamp <= deadline, "expired");
```

Team Response

Fixed.

`previewRedeem()` Can Return Incorrect Quotes for Out-of-Range Share Inputs

Description

`previewRedeem()` does not check that the input `shares` is less than or equal to `totalSupply()`. For very large out-of-range inputs, its internal `uint128` cast can silently truncate and return an incorrect redemption quote.

`previewRedeem()` calculates the amount of liquidity to quote and then directly casts it to `uint128`. Unlike `redeem()`, this function accepts any arbitrary `shares` value from the caller. If the calculated liquidity exceeds `type(uint128).max`, the explicit cast does not revert and instead truncates the value. This can make the function return a wrong quote, including `[0, 0]` for some very large inputs.

Location of Affected Code

File: [contracts/VltUsdcVault.sol](#)

```
uint128 liquidityToRemove = uint128((uint256(positionLiquidity()) * shares) / supply);
```

Impact

This has no direct effect on vault funds or accounting because `previewRedeem()` is only a view function. The impact is limited to off-chain display or integration errors, where a UI or dashboard using an unsanitized out-of-range input may show an incorrect redemption estimate.

Recommendation

Restrict the function to its intended input range before calculating liquidity:

```
require(shares <= totalSupply(), "shares-exceeds-supply");
```

This prevents silently wrapped quotes and makes invalid preview inputs fail clearly.

Team Response

Fixed.

Position Key Is Recomputed Instead of Cached

Description

The vault recomputes the same Uniswap V4 position key in multiple functions even though all inputs are fixed for the lifetime of the contract.

`positionKey` is derived from `address(this)`, `tickLower`, `tickUpper`, and `bytes32(0)`. These values do not change after deployment, but the contract recalculates the key each time. This appears in `positionLiquidity()` and `compoundClaimable()`, and `positionLiquidity()` is used by several deposit, redeem, compound, and view paths.

Location of Affected Code

File: [contracts/VltUsdcVault.sol](#)

```
bytes32 positionKey = Position.calculatePositionKey(  
    address(this),  
    tickLower,  
    tickUpper,  
    bytes32(0)  
);
```

Impact

This has no security or correctness impact. It only wastes a small amount of gas by repeating the same keccak256 calculation.

Recommendation

Cache the position key once in the constructor as an immutable `bytes32` and reuse it wherever needed.

Team Response

Fixed.

recipient == vault Permanently Locks Minted Shares

Description

`ZapHelper.zapDeposit()` forwards the user-supplied `recipient` directly to `VltUsdcVault.deposit()`. If `recipient` is set to the vault address, the vault mints shares to itself. Since `redeem()` only burns `msg.sender`'s shares and the vault has no function that can redeem or transfer its own shares, those shares become permanently inaccessible.

In `ZapHelper.zapDeposit()`, `recipient` is forwarded without validation, and the vault only rejects the zero address. If `recipient == address(vault)`, the shares are minted to the vault contract itself. The only redemption path burns `msg.sender`'s shares:

```
function redeem(uint256 shares, address receiver) external {  
    ...  
    _burn(msg.sender, shares);  
}
```

Because the vault cannot call `redeem()` on itself and has no rescue, sweep, or admin transfer function, the minted shares cannot be recovered.

Location of Affected Code

File: [contracts/ZapHelper.sol](#)

```
shares = IVltUsdcVault(vault).deposit(  
    vltOut,  
    usdcForLp,  
    minShares,  
    deadline,  
    recipient  
);
```

File: [contracts/VltUsdcVault.sol](#)

```
require(recipient != address(0), "zero-recipient");  
...  
_mint(recipient, shares);
```

Impact

A caller or integrating frontend that accidentally sets `recipient` to the vault address causes the deposited value to be represented by shares permanently held by the vault. The user receives no redeemable shares and loses access to the deposited funds. This is a caller/integration footgun, not a third-party theft vector.

Recommendation

Reject the vault address as a share recipient in `zapDeposit()` and/or `deposit()`:

```
require(recipient != vault, "bad-recipient");
```

For stronger defence, the vault can also reject `recipient == address(this)` directly inside `deposit()`.

Team Response

Fixed.

FINDING I-11

Missing Constructor Validation Can Permanently Misconfigure the Vault

INFO

FIXED

Description

`VltUsdcVault` accepts critical deployment parameters without fully validating them. The vault is immutable and ownerless, so a wrong `PoolManager`, pool key, fee tier, tick spacing, native token leg, or incorrectly labelled USDC token cannot be corrected after deployment.

The constructor only checks that hooks are disabled, token ordering is correct, and `_usdc` is one of the two pool currencies. It does not verify that `_poolManager` is the expected canonical Uniswap V4 `PoolManager`, that the pool is initialized, that the pool fee is the intended `10000` (1%) tier, that `tickSpacing > 0`, or that the token legs match the expected USDC/VLT behavior.

This means a deployment with `tickSpacing == 0` reverts with a raw panic, while other bad parameters may deploy successfully and only fail or behave incorrectly later. `unlockCallback` also trusts only the configured `poolManager`, so a malicious or wrong manager address would be fully trusted.

Location of Affected Code

File: [contracts/VltUsdcVault.sol](#)

```
require(address[_key.hooks] == address[0], "hooks-not-allowed");
require(Currency.unwrap[_key.currency0] < Currency.unwrap[_key.currency1], "token-order");
```

```
poolManager = _poolManager;  
poolKey = _key;  
  
require(!_usdc == t0 || !_usdc == t1, "usdc-not-in-pool");  
  
int24 spacing = _key.tickSpacing;  
tickLower = (TickMath.MIN_TICK / spacing) * spacing;  
tickUpper = (TickMath.MAX_TICK / spacing) * spacing;
```

Impact

A malicious PoolManager could fabricate callback behavior and cause severe loss, but that requires the trusted deployer to intentionally or accidentally deploy against the wrong manager.

Recommendation

Add constructor checks for the expected PoolManager address, initialized pool state, `fee == 10000`, `tickSpacing > 0`, non-native token legs, and expected USDC decimals. Keep the deploy-script checks, but enforce the critical assumptions on-chain as well.

Team Response

Fixed.



Your security partner, for the long term

[SHIELDIFY.ORG](https://shieldify.org) · [X.COM/SHIELDIFYSEC](https://x.com/shieldifysec) · [LINKEDIN](#)